Deep Learning

LeCun, Bengio & Hinton (Nature, 2015)

Your Name

STAT 3494W

April 24, 2026

Outline

1. Introduction
2. The Problem
3. Backpropagation
4. History
5. Convolutional Neural Networks
6. Language Processing
7. How the Paper Holds Up

Why This Paper?

- Published in *Nature* — most prestigious scientific journal
- **~70,000 citations** — one of the most cited CS papers ever
- Written by the three researchers most responsible for the field
- All three: **2018 Turing Award** (Nobel Prize of CS)
- Hinton: **2024 Nobel Prize in Physics**

The paper is simultaneously:

- A technical survey
- A historical account written by its participants
- An argument to a skeptical research community

The Authors: Geoffrey Hinton

- Popularized **backpropagation** (1986)
- **Boltzmann Machines** (1985)
- Central to the 2006 CIFAR deep learning breakthrough
- Nobel Prize in Physics 2024

The Authors: Yann LeCun

- Invented **Convolutional Neural Networks** (LeNet, late 1980s)
- CNN check-reading system processed **¿10% of all US checks** by late 1990s
- Chief AI Scientist at Meta

The Authors: Yoshua Bengio

- Pioneered **word embeddings** and neural language models
- **Most-cited computer scientist** in the world
- By both total citations and h-index

Note: throughout the paper, breakthroughs their own groups produced are described without attribution. Watch for it.

The Dominant Paradigm Before Deep Learning

Hand-engineered features:

- A human expert decides what the machine should measure
- A **linear classifier** computes weighted sum against threshold

Example: HOG (Histogram of Oriented Gradients)

1. Compute edge directions at each pixel
2. Build histograms per cell
3. Feed resulting vector to SVM

Every new problem needed new hand-crafted features

Linear vs Learned Decision Boundaries

Classic SVM

[Linear hyperplane]

Deep Learning

[Nuanced, curved boundary]

Deep learning learns **non-linear transformations** that make complex boundaries possible

The Fundamental Limit

Known since the 1960s:

- Linear classifiers can only partition space into **half-spaces separated by a hyperplane**
- Complex, curved, or nested decision boundaries are impossible
- No matter the scale, there are functions a linear classifier cannot learn

The question:
What if the machine learned the features itself?

What is Deep Learning?

Deep learning = representation learning with **multiple levels of abstraction**

- Neural networks with many layers ("deep")
- Each layer transforms the representation
- Higher layers = more abstract features

Key insight:

"Deep learning methods are representation-learning methods with multiple levels of representation, obtained by composing simple but non-linear modules."

Representation Learning

"Representation learning is a set of methods that allows a machine to be fed with raw data and to automatically discover the representations needed for detection or classification."

- Rather than hand-crafting features, **train the system to discover them**
- Each layer learns increasingly abstract representations:

```
Raw pixels → Edges → Shapes → Parts → Objects
```

Why Depth Matters

Two compounding advantages:

1. **Combinatorial capacity**
   - With n binary features: 2^n possible configurations
   - 30 neurons can distinguish over 1 billion concepts
2. **Exponential expressiveness**
   - Each additional layer **multiplies** expressive power
   - A shallow network needs exponentially more neurons to match a deep one

Depth is a qualitative difference, not merely quantitative

Feedforward Neural Network Structure

[Feedforward NN diagram with labeled weights w_{ij} and biases b_j]

Each neuron computes:

$$z_j = \sum_i w_{ij} x_i + b_j \quad \text{then} \quad a_j = \sigma(z_j)$$

where σ is the activation function

Backpropagation: The Algorithm

The training algorithm that makes representation learning practical

1. **Forward pass:** compute prediction $\hat{y}$
2. **Compute loss:** $L = \frac{1}{2}(y - \hat{y})^2$
3. **Backward pass:** compute gradients via chain rule
4. **Update weights:** $w \leftarrow w - \eta \frac{\partial L}{\partial w}$
5. Repeat

Backpropagation: The Math

Chain rule propagates error backward:

For weight w_{ij} connecting layer i to layer j:

$$\frac{\partial L}{\partial w_{ij}} = \frac{\partial L}{\partial a_j} \cdot \frac{\partial a_j}{\partial z_j} \cdot \frac{\partial z_j}{\partial w_{ij}}$$

Key insight: Each term is local

- $\frac{\partial z_j}{\partial w_{ij}} = a_i$ (input from previous layer)
- $\frac{\partial a_j}{\partial z_j} = \sigma'(z_j)$ (derivative of activation)
- $\frac{\partial L}{\partial a_j}$ propagates from layers above

This is why activation functions must be differentiable

Stochastic Gradient Descent (SGD)

Why "stochastic"?

Batch gradient descent:

$$w \leftarrow w - \eta \frac{1}{N} \sum_{i=1}^{N} \nabla_w L_i$$

Compute gradient over *all* N training examples — slow!

Stochastic gradient descent:

$$w \leftarrow w - \eta \nabla_w L_i \quad \text{(single random example)}$$

Mini-batch SGD: Use small random subset (e.g., 32 examples)

- Noisy estimate of true gradient
- But much faster iteration
- Noise can help escape local minima

Activation Functions

Why do we need them?

- Without non-linearity, stacked layers collapse to single linear transform
- Non-linear σ enables learning complex functions

Sigmoid / Tanh:

- Smooth, differentiable
- Problem: cause **vanishing gradients** — $\sigma'(z) \to 0$ for large $|z|$

ReLU ($\max(0, x)$):

- Simple, fast to compute
- Gradient is 1 for $x > 0$ — no vanishing
- The paper's recommended default

Backpropagation: Code Example

[Code: simple 2-layer network with ReLU, forward/backward pass]

The 1990s Objection

The prevailing concern:

- Backpropagation would get trapped in poor **local minima**
- Points where gradient is zero but solution is far from optimal
- Widely cited as proof deep networks could never work

The paper's rebuttal:

> *"The landscape is packed with a combinatorially large number of saddle points where the gradient is zero, and the surface curves up in most dimensions and curves down in the remainder."*

Saddle points are not local minima — there is always a descent direction

What Do Feedforward Networks Learn?

[Visualization: weight matrices from feedforward NN on images]

Observation: Weights appear relatively unstructured

- No clear feature detectors
- Contrast with CNNs which learn interpretable filters

This motivates architectures with built-in structure...

The AI Winters

Period	Event
1950s–60s	Symbolic AI, first wave of optimism
1970s	**AI Winter 1** — symbolic approaches hit limits
1980s	Expert systems — encode human knowledge explicitly
1990s	**AI Winter 2** — expert systems too brittle
1986	Hinton's backprop paper — *largely ignored*
1990s	LeCun's CNNs deployed — *largely ignored*
2000s	SVMs and kernel methods dominate mainstream

Deep learning survived two cycles of hype and abandonment
The authors lived through both winters, continued regardless

The 2006 Breakthrough: The Problem

Training deep networks was extremely difficult:

- No ReLU — stuck with sigmoid/tanh
- Poor weight initialization
- No dropout regularization
- Gradients vanished before reaching early layers
- Limited labeled data
- Limited compute power

Boltzmann Machines

[Diagram: Boltzmann Machine structure]

Hinton & Sejnowski, 1985

- Stochastic recurrent neural network
- Can learn internal representations without labels
- **Restricted Boltzmann Machines (RBMs):** no intra-layer connections

Foundation for the 2006 breakthrough

The 2006 Breakthrough: The Solution

Greedy layer-wise pretraining:

1. Train each layer independently using **unlabeled data**
2. Use Restricted Boltzmann Machines
3. Stack the pretrained layers
4. Fine-tune end-to-end with labeled data

Why it mattered:

- Proved deep nets could learn features **without labels**
- Proved pursuing depth was worthwhile at all

Note: the paper attributes this to "a group at CIFAR." Hinton and LeCun were that group.

First Commercial Payoff: Speech Recognition

- Adopted by major industry groups ~2009
- Deployed in Android phones by 2012
- Compounded by rapidly improving GPU hardware

GPUs: Originally for graphics/gaming

- Massively parallel architecture
- Perfect for matrix operations in neural networks
- Training times dropped from weeks to hours

Once training innovations arrived, pretraining became unnecessary for large labeled datasets

The Problem with Fully Connected Networks

Applying standard networks to images:

- A 256×256 RGB image = **~200,000 inputs**
- Fully connected to 1000 neurons = 200 million parameters in layer 1 alone
- No spatial awareness — pixel (0,0) and pixel (255,255) treated the same
- Small shifts break everything

Unmanageable and fundamentally wrong approach

CNN Structure

[Diagram: CNN architecture with labeled components]

Key components:

- **Convolutional layers:** learnable filters/kernels
- **Pooling layers:** downsample spatial dimensions
- **Fully connected:** classification at the end

The CNN Solution: Key Ideas

1. **Local connectivity**
   - Small filter (e.g., 3×3) slides across the image
   - Detects local patterns: edges, textures, curves
2. **Weight sharing**
   - Same filter weights applied at every position
   - Drastically reduces parameters
   - Translation invariance built in
3. **Pooling**
   - Downsample and retain dominant signal
   - Reduces spatial dimensions progressively

The kernel weights are learned during training

What Do Learned Kernels Look Like?

[Visualization: learned kernels from CNN layer 1]

First layer kernels typically learn:

- Edge detectors (horizontal, vertical, diagonal)
- Color blobs
- Gabor-like filters

These emerge from training — not hand-designed

Kernels Applied to Images

[Visualization: kernel convolution on sample image → feature maps]

Each kernel produces a feature map:

- Edge kernel → highlights edges in image
- Color kernel → highlights color regions
- Stacking layers builds more complex features

CNNs Learn a Hierarchy

Layer 1: Edges, gradients, colors
Layer 2: Corners, textures, simple shapes
Layer 3: Parts — eyes, wheels, windows
Layer 4+: Objects — faces, cars, buildings

```
Pixels → Edges → Shapes → Parts → Objects
```

This structure emerges from training — not programmed

The Biological Parallel

"When ConvNet models and monkeys are shown the same picture, the activations of high-level units in the ConvNet explains half of the variance of random sets of 160 neurons in the monkey's inferotemporal cortex."

- CNNs mirror the hierarchical structure of biological vision
- Not designed to — emerged from learning
- The visual cortex processes: edges $\rightarrow$ shapes $\rightarrow$ objects

CNNs: Commercial History

LeCun's work (described without his name):

- Time-delay neural networks for speech recognition (early 1990s)
- Check reading: **¿10% of all US checks** by late 1990s
- Microsoft OCR and handwriting recognition
- Face and hand detection in natural images

Then:

> *"ConvNets were largely forsaken by the mainstream computer-vision and machine-learning communities until the ImageNet competition in 2012."*

Working for 20 years. Mainstream ignored it.

ImageNet 2012: AlexNet

The result that changed everything:

- Top-5 error: **15.3%** vs runner-up's **26.2%**
- Not incremental — almost halved the error rate

Key ingredients:

- GPUs (trained on 2 GTX 580s)
- ReLU activation
- Dropout regularization
- Data augmentation

The field could no longer dismiss deep learning

The Paper's Forward Look (2015)

"Companies such as Mobileye and NVIDIA are using ConvNet-based methods in their upcoming vision systems for cars."

What happened:

- Tesla Autopilot launched October 2015
- NVIDIA pivoted to AI — stock up ~20,000% since 2015
- Every major tech company adopted deep learning

The hype was warranted — they knew it would work

The Old Approach: N-grams

How NLP worked before deep learning:

- Count frequencies of word sequences of length N
- Predict next word by frequency alone
- Possible sequences scale as V^N where V = vocabulary size

The problems:

- Hopelessly unscalable
- No semantic understanding
- "fast car" and "quick car" are completely unrelated
- Fundamentally counting, not understanding

Word Embeddings: The Insight

Bengio et al.'s approach:

1. Represent each word as **one-hot vector** (one component $= 1$, rest $= 0$)
2. First layer learns to map these into **dense embeddings**
3. Semantically similar words end up geometrically close

This structure was not programmed — it emerged:

> *"When trained to predict the next word in a news story, the learned word vectors for Tuesday and Wednesday are very similar, as are the word vectors for Sweden and Norway."*

Word Embeddings: Why It's Surprising

It works on:

- Small, clean datasets (Hinton's 1986 family trees)
- Large, messy real-world corpora

This was not the expected result at the time

Foreshadows:

- Word2Vec (2013)
- GloVe (2014)
- Every modern language model embedding

Recurrent Neural Networks: The Problem

Feedforward networks for language:

- Stateless — each input processed independently
- No memory of prior context
- Cannot model sequence, grammar, long-range dependencies

Language is inherently sequential

"The cat sat on the ___" requires knowing what came before

RNN: Unrolled Structure

[Diagram: RNN unrolled through time]

Key insight: The **same weights** are used at every time step

$$h_t = \sigma(W_{hh} h_{t-1} + W_{xh} x_t + b)$$

This weight sharing is both the power and the problem...

RNNs: What They Enable

Key idea: Pass a hidden state forward through the sequence

- Network retains implicit memory of previous inputs
- Enables sentence-level context
- Sequence-to-sequence tasks become possible

Example: Machine translation

- Translate English $\rightarrow$ French
- Learns implicit grammatical and semantic structure
- Not word-by-word lookup

The Vanishing Gradient Problem

Backpropagating through time:

$$\frac{\partial L}{\partial h_1} = \frac{\partial L}{\partial h_T} \cdot \prod_{t=2}^{T} \frac{\partial h_t}{\partial h_{t-1}}$$

- Each $\frac{\partial h_t}{\partial h_{t-1}}$ involves W_{hh} and σ'
- **Same weights multiplied T times**
- If $\|W_{hh}\| < 1$: gradients **vanish** exponentially
- If $\|W_{hh}\| > 1$: gradients **explode**

Result: Long-range dependencies cannot be learned

"The cat, which sat on the mat that belonged to my neighbor, ___"

LSTM: Structure

Hochreiter & Schmidhuber, 1997

[Diagram: LSTM cell with gates labeled]

Key innovation: Separate **cell state** C_t from hidden state h_t
Cell state flows through with minimal transformation — gradients preserved

LSTM: The Gates

Forget gate: What to discard from cell state

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Input gate: What new information to store

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

Output gate: What to output

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

Result: Network learns what to remember and what to forget

LSTM: Better, But Not Perfect

LSTM solves vanishing gradients — mostly

- Gradients can flow across hundreds of time steps
- Much better than vanilla RNN

But still has limitations:

- Doesn't remember forever — gradients still decay, just slower
- Sequential processing — can't parallelize across time steps
- Fixed-size hidden state — information bottleneck

These limitations motivate the search for better architectures...

The Memory Bottleneck

The remaining structural problem:

- Even LSTM compresses everything into fixed-size hidden state
- Real memory is addressable, structured, large
- A single vector is insufficient

Neural Turing Machines (DeepMind, 2014):

- RNN controller + external addressable memory bank
- Reads/writes via **attention mechanism**
- Differentiable approximation of a Turing machine

The Paper's Prediction

> *"We expect systems that use RNNs to understand sentences or whole documents will become much better when they learn strategies for selectively attending to one part at a time."*

Two years later:

Vaswani et al., *"Attention Is All You Need"* (2017)

- Dispenses with recurrence entirely
- Attention alone handles memory and context
- **The transformer is born**

The Timing of This Paper

2015 — A pivotal moment:

- 3 years after AlexNet shocked ImageNet (2012)
- Deep learning finally has mainstream attention
- But many researchers still skeptical — remember the winters
- 2 years *before* "Attention Is All You Need" (2017)

The authors are making a case:

- This is real, this is working, this will continue
- Here's the technical foundation
- Here's where it's going

Prediction: Unsupervised Learning

"Unsupervised learning had a catalytic effect in reviving interest in deep learning, but has since been overshadowed by the successes of purely supervised learning. Although we have not focused on it in this Review, we expect unsupervised learning to become far more important in the longer term."

"Human and animal learning is largely unsupervised: we discover the structure of the world by observing it, not by being told the name of every object."

Key points:

- Supervised learning dominated at the time
- They predict unsupervised will become more important
- Biological motivation: humans learn without labels

Prediction: Vision + Reinforcement Learning

"Human vision is an active process that sequentially samples the optic array in an intelligent, task-specific way using a small, high-resolution fovea with a large, low-resolution surround."

"We expect much of the future progress in vision to come from systems that are trained end-to-end and combine ConvNets with RNNs that use reinforcement learning to decide where to look."

Key points:

- Vision is active, not passive
- CNN + RNN + RL for "where to look"
- Systems already outperform passive vision at classification

Prediction: NLP + Attention

"Natural language understanding is another area in which deep learning is poised to make a large impact over the next few years."

"We expect systems that use RNNs to understand sentences or whole documents will become much better when they learn strategies for selectively attending to one part at a time."

Key points:

- NLP will see "large impact" in coming years
- Attention is the key missing mechanism
- Selective focus on "one part at a time"

Prediction: Reasoning

"Ultimately, major progress in artificial intelligence will come about through systems that combine representation learning with complex reasoning."

"Although deep learning and simple reasoning have been used for speech and handwriting recognition for a long time, new paradigms are needed to replace rule-based manipulation of symbolic expressions by operations on large vectors."

Key points:

- Deep learning alone is not enough
- Must combine with "complex reasoning"
- New paradigms needed — not just scaling

The Scorecard

Prediction (2015)	**Outcome**
Unsupervised learning will dominate	✓(GPT, BERT)
Attention is the key mechanism	✓✓(Transformers)
CNN + RNN + RL for active vision	✓(DeepMind)
Need representation + reasoning	Open problem

- Near-term predictions: vindicated within 2–5 years
- The one unsolved problem they identified remains central

The Philosophical Frame

A distinction from cognitive science:

Fast inference:

- Automatic, pattern-based, intuitive
- Recognizing a face, understanding speech

Deliberate reasoning:

- Slow, logical, symbolic
- Multi-step proofs, causal inference

Deep learning models the former — and does so better than anything prior

The authors understood clearly what deep learning was *and what it was not*

Conclusion

What makes this paper special:

- Not just a survey — a **persuasion document**
- Written by the people who **built the field**
- Every claim backed by work they personally did
- Published at the right moment — deep learning proven but not yet dominant

The result:

- ~70,000 citations
- Every near-term prediction came true
- The open question they identified remains open

Closing

- Three researchers
- Thirty years of work dismissed twice by the mainstream
- One paper in *Nature*, 2015

The world they described is the one we are living in.

Questions?

Thank you

Reference:

- LeCun, Bengio, Hinton. "Deep Learning." *Nature* 521, 2015.